

Report – Python4Physics 2025

University of California Berkeley, USA

Shah Nawaz Ali

alis07655@gmail.com

Introduction

The 2025 Python4Physics program conducted at UC Berkeley was a unique experience that merged the power of computational thinking with the elegance of Laws of Physics. In ten intensive days, we studied and worked on the topics ranging from particle physics to astronomy, using Python as our main computational tool. We became familiar with Python libraries like Numpy, Matplotlib and Scipy. The series of lectures, coding projects, and interactions with researchers created a deep and practical understanding of how modern physics problems can be solved numerically.

Highlights of the programme

Days 1–2: Python Fundamentals and Introductory Projects

We began the programme with immersive hands-on exploration of Python fundamentals including basic operators, logic and control flow structures. This foundational proficiency

allowed us to experiment with introductory Physics simulations, leading into our first projects on applied problem-solving.

Day 3: Statistics in Physics

Guest lecturer Kenneth McElvain emphasized the critical role of statistical methods in experimental physics. We explored key concepts such as probability distributions, standard deviation, and histogram construction, which are essential tools for the rigorous analysis of empirical data.

Day 4: Astrophysics and Particle Decay

We deeply explored the Cosmic Microwave Background (CMB) and astrophysical experiments conducted at the South Pole. Building on this foundation, we developed a computational simulation of neutron decay that rigorously applied the conservation of momentum and energy, thereby translating foundational theoretical principles into precise numerical codes.

Days 5–6: Resonances and Gravitational Waves

In the lectures by Livia Belman-Wells and Professor Xu, we examined the Breit-Wigner distribution and resonance phenomena, and learned curve fitting techniques using Python's SciPy library. Then we applied these methods to match gravitational wave templates to simulated data, mirroring the analytical approaches used by LIGO physicists in real-world detection.

Day 7: Ising Model and Spin Systems

Felipe Ortega Gama's lecture on magnetism and the Ising model provided a rigorous framework for simulating spin interactions, incorporating both nearest- and next-nearest-neighbor couplings. This approach facilitated a deeper exploration of fundamental concepts in

condensed matter physics, including symmetry breaking, degeneracy, and the emergent collective behavior of spin systems.

Days 8–9: Calculus and the Standard Model

We studied differential and integral calculus through both theoretical analysis and computational implementation, deepening our understanding of continuous systems. We also delved into the Standard Model of Particle Physics, examining the fundamental constituents of matter—quarks and leptons—as well as force-mediating bosons and the role of spontaneous symmetry breaking via the Higgs mechanism.

Day 10: Axions and Dark Matter

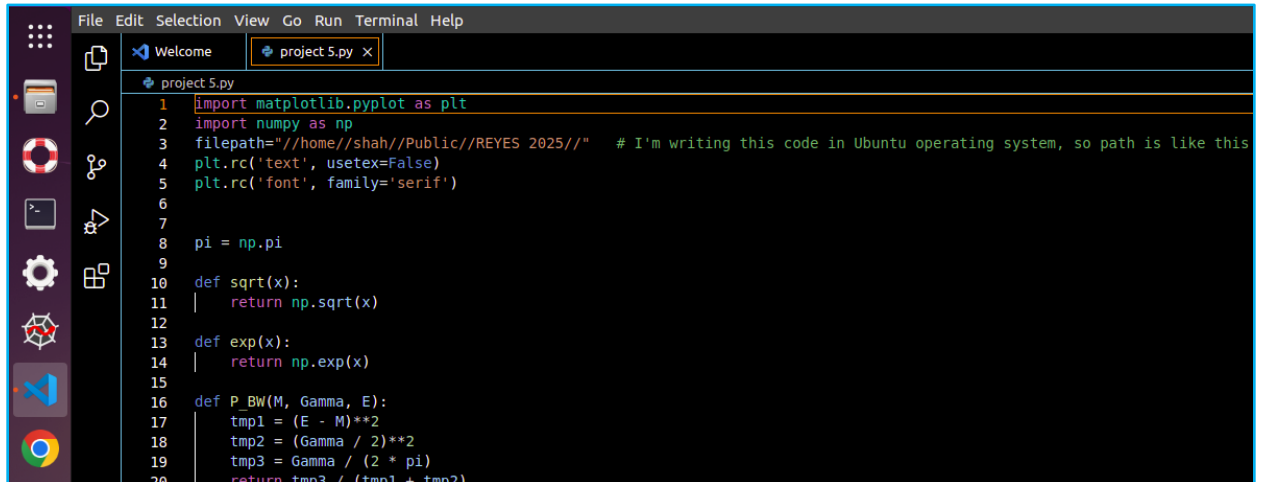
The final lecture by Prof. Chiara Salemi explored dark matter candidates such as the axion and the experimental challenges of detecting these extremely weakly interacting particles. It elegantly connected cosmic-scale phenomena with quantum-scale theories, bridging two frontiers of physics: the cosmos and the quantum realm.

Personal Reflections

The Python4Physics program was incredibly insightful. It went beyond just teaching Python syntax—it demonstrated how computation can be a powerful tool for discovery in physics. Through the program, I developed a deeper appreciation for statistical inference, numerical modeling, and the theoretical frameworks that underpin various branches of physics. One of my favorite projects was simulating the Ising model, which helped me understand how microscopic states give rise to macroscopic magnetism. Another highlight was modeling gravitational waves, where we matched simulated data to theoretical waveforms—a fascinating blend of computation and theory.

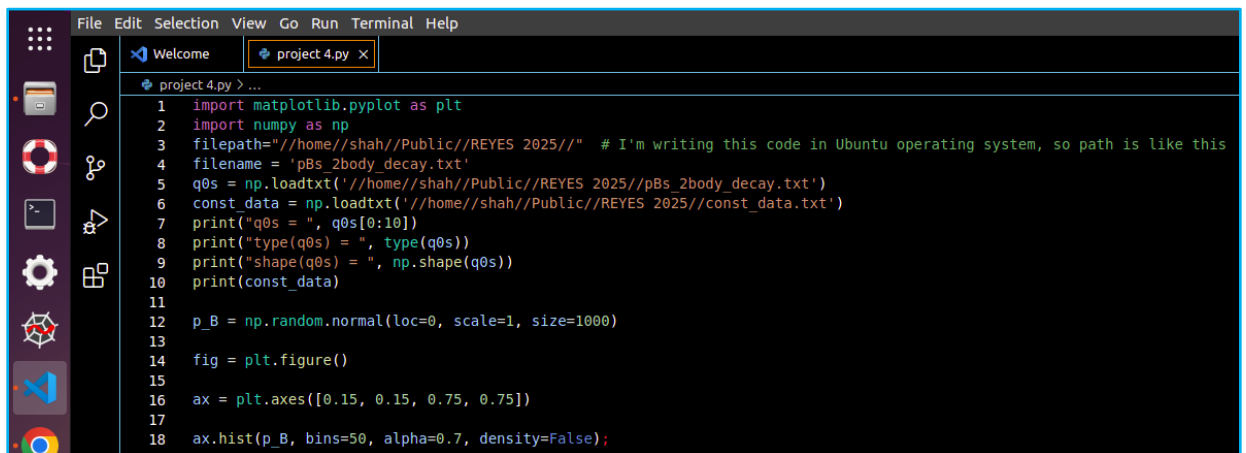
Code Excerpts

1. Curve Fitting (Breit-Wigner Distribution):

A screenshot of a code editor window titled 'project 5.py'. The editor shows Python code for curve fitting using the Breit-Wigner distribution. The code imports matplotlib.pyplot as plt and numpy as np. It defines a function sqrt(x) and a function exp(x). The main function P_BW(M, Gamma, E) calculates the Breit-Wigner distribution. The code is as follows:

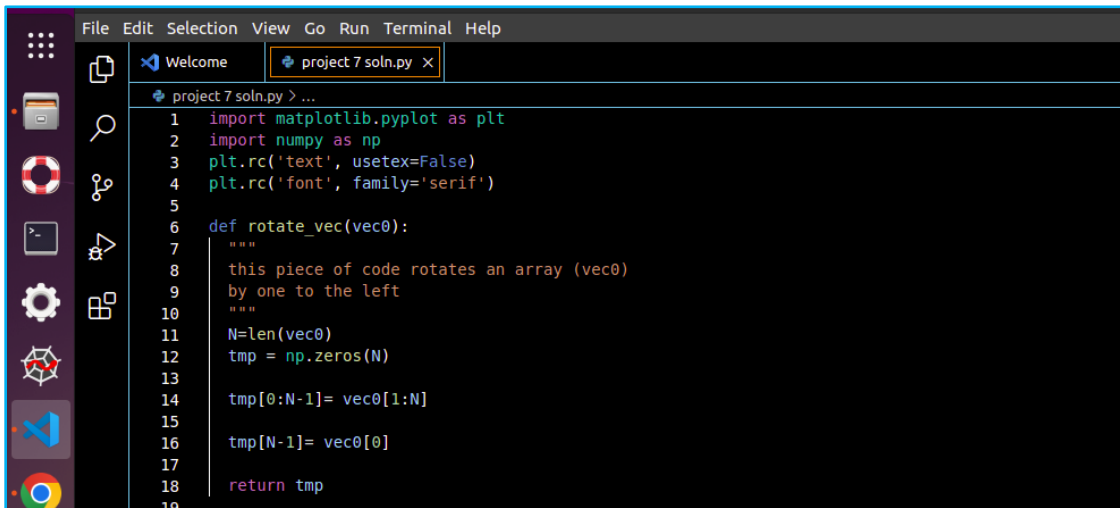
```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 filepath="//home//shah//Public//REYES 2025//" # I'm writing this code in Ubuntu operating system, so path is like this
4 plt.rc('text', usetex=False)
5 plt.rc('font', family='serif')
6
7
8 pi = np.pi
9
10 def sqrt(x):
11     return np.sqrt(x)
12
13 def exp(x):
14     return np.exp(x)
15
16 def P_BW(M, Gamma, E):
17     tmp1 = (E - M)**2
18     tmp2 = (Gamma / 2)**2
19     tmp3 = Gamma / (2 * pi)
20     return tmp3 / (tmp1 + tmp2)
```

2. Particle Decay Simulation:

A screenshot of a code editor window titled 'project 4.py'. The editor shows Python code for a particle decay simulation. The code imports matplotlib.pyplot as plt and numpy as np. It defines a function sqrt(x) and a function exp(x). The main function P_BW(M, Gamma, E) calculates the Breit-Wigner distribution. The code is as follows:

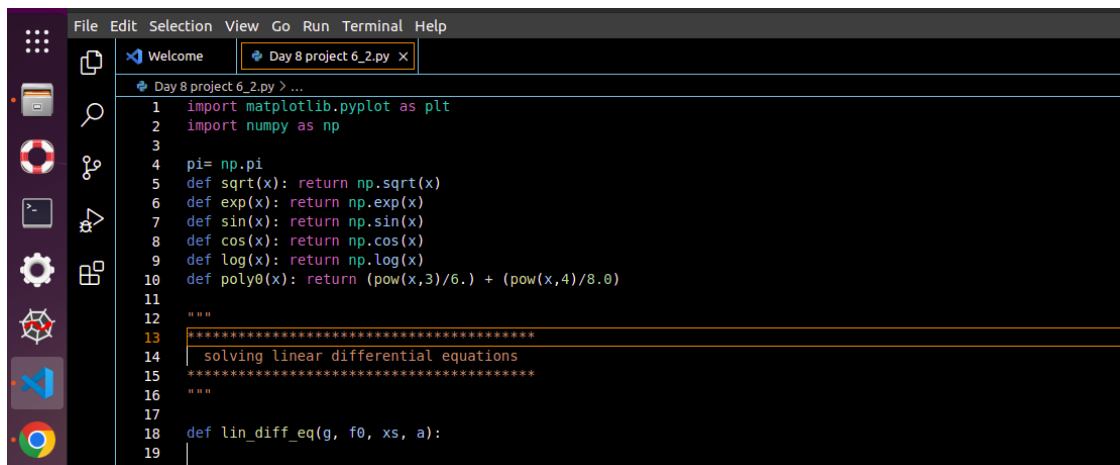
```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 filepath="//home//shah//Public//REYES 2025//" # I'm writing this code in Ubuntu operating system, so path is like this
4 filename = 'pBs_2body_decay.txt'
5 q0s = np.loadtxt('//home//shah//Public//REYES 2025//pBs_2body_decay.txt')
6 const_data = np.loadtxt('//home//shah//Public//REYES 2025//const_data.txt')
7 print("q0s = ", q0s[0:10])
8 print("type(q0s) = ", type(q0s))
9 print("shape(q0s) = ", np.shape(q0s))
10 print(const_data)
11
12 p_B = np.random.normal(loc=0, scale=1, size=1000)
13
14 fig = plt.figure()
15
16 ax = plt.axes([0.15, 0.15, 0.75, 0.75])
17
18 ax.hist(p_B, bins=50, alpha=0.7, density=False);
```

3. Ising Model Energy Calculation:



```
File Edit Selection View Go Run Terminal Help
Welcome project 7 soln.py x
project 7 soln.py > ...
1 import matplotlib.pyplot as plt
2 import numpy as np
3 plt.rc('text', usetex=False)
4 plt.rc('font', family='serif')
5
6 def rotate_vec(vec0):
7     """
8     this piece of code rotates an array (vec0)
9     by one to the left
10    """
11    N=len(vec0)
12    tmp = np.zeros(N)
13
14    tmp[0:N-1]= vec0[1:N]
15
16    tmp[N-1]= vec0[0]
17
18    return tmp
19
```

4. Numerically solving Differential Equation:



```
File Edit Selection View Go Run Terminal Help
Welcome Day 8 project 6_2.py x
Day 8 project 6_2.py > ...
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 pi= np.pi
5 def sqrt(x): return np.sqrt(x)
6 def exp(x): return np.exp(x)
7 def sin(x): return np.sin(x)
8 def cos(x): return np.cos(x)
9 def log(x): return np.log(x)
10 def poly0(x): return (pow(x,3)/6.) + (pow(x,4)/8.0)
11
12 """
13 *****
14 solving linear differential equations
15 *****
16 """
17
18 def lin_diff_eq(g, f0, xs, a):
19
```

Conclusion:

The 2025 Python4Physics programme gave me both the tools and the inspiration to further pursue computational physics. I developed stronger coding skills, a broader interest in Physics and a renewed passion for scientific discovery. I am grateful to the UC Berkeley, mentors and fellow students who made this such a rewarding experience.